LAPORAN PRAKTIKUM PEMROGRAMAN WEB
MODUL 7: CRUD DAN UPLOAD FILE PADA LARAVEL



Disusun Oleh:

Nama: Andhika Yuridho

NIM: 707012500059

Kelas: D4SIKC-49-04

Mata Kuliah: Pemrograman Web

UNIVERSITAS TELKOM

FAKULTAS ILMU TERAPAN

JURUSAN S1 TERAPAN SISTEM INFORMASI KOTA CERDAS

TAHUN AJARAN 2026

1. Tujuan

Tugas akhir Modul 7 yang saya kerjakan adalah soal nomor 1 pada bagian Tugas Praktikum, yaitu aplikasi LaporBanjir untuk BPBD Kabupaten Bandung. Tujuan pengerjaannya adalah sebagai berikut:

1. Membangun aplikasi LaporBanjir dari project Laravel baru dengan Laravel Breeze sebagai dasar authentication.

2. Membuat tabel laporan yang memuat identitas pelapor, lokasi, kecamatan, tinggi air, keterangan, foto, dan status penanganan.

3. Menerapkan upload foto bukti pada form pelaporan warga beserta validasi jenis dan ukuran berkas.

4. Menyimpan foto di storage/app/public dan menyajikannya kembali melalui symbolic link public/storage.

5. Membangun pengelolaan laporan untuk petugas memakai resource controller dan Route::resource di dalam middleware auth.

6. Memastikan foto lama terhapus ketika diganti dan foto ikut terhapus ketika laporan dihapus.

7. Menampilkan pesan berhasil dan pesan galat validasi pada setiap proses tambah, ubah, dan hapus.

2. Alat dan Bahan

Praktikum ini dikerjakan pada satu laptop Lenovo 83DV dengan prosesor Intel Core i7-13650HX (14 core, 20 thread), RAM 12 GB, dan Windows 11 Pro versi 10.0.26100. Perangkat lunak yang dipakai adalah Laragon sebagai server lokal dengan Apache, MySQL 8.4.3, dan phpMyAdmin 5.2.3; PHP 8.3.30 (ZTS); Composer 2.10.3; Laravel Framework 13.38.0; Laravel Breeze 2.4; Node.js 24.22.1 dan npm 11.19.1; Vite 8.3.5 dengan Tailwind CSS 3.4.19; Visual Studio Code 1.143.1; serta Google Chrome untuk pengujian.

Aplikasi memakai database MySQL db_laporbanjir. Berkas uji yang dipakai adalah tiga gambar JPG berukuran 78 KB sampai 1,6 MB, satu arsip ZIP, dan satu gambar 6 MB untuk menguji batas ukuran unggahan. Server pengembangan dijalankan dengan php artisan serve pada alamat http://127.0.0.1:8125.

3. Langkah-Langkah Praktikum

Pengerjaan dimulai dari project Laravel baru, lalu dilanjutkan dengan pembuatan tabel, controller, route, dan view. Bagian pengelolaan laporan memakai Laravel Breeze karena authentication sudah tersedia dan saya hanya perlu menambahkan satu menu pada navigasi. Seluruh perintah dijalankan dari terminal di dalam folder project.

3.1 Menyiapkan Project LaporBanjir dengan Laravel Breeze

Project dibuat dengan composer create-project laravel/laravel praktikum7-ta. Nama foldernya saya sesuaikan menjadi praktikum7-ta mengikuti folder tugas, sedangkan nama aplikasi pada file .env diisi LaporBanjir. Konfigurasi database diubah menjadi DB_CONNECTION=mysql, DB_HOST=127.0.0.1, DB_PORT=3306, DB_DATABASE=db_laporbanjir, DB_USERNAME=root, dan DB_PASSWORD kosong.

Laravel Breeze dipasang dengan composer require laravel/breeze --dev lalu php artisan breeze:install blade. Perintah kedua menyalin route, controller, dan view untuk register, login, logout, lupa password, dan profil, serta menjalankan npm install dan npm run build. Setelah itu php artisan migrate membuat tabel bawaan Laravel dan php artisan storage:link menghubungkan public/storage ke storage/app/public.

Ada satu penyesuaian nama tabel. Database db_laporbanjir sudah dipakai pada tugas Modul 6 dan masih menyimpan tabel laporan dengan skema lama. Karena tugas ini meminta aplikasi yang dibangun dari awal dengan kolom yang berbeda, tabel barunya saya beri nama laporans mengikuti bentuk jamak bawaan Laravel. Tabel laporan dari tugas sebelumnya tidak saya ubah.

3.2 Membuat Migration dan Model Laporan

Tabel dibuat dengan php artisan make:model Laporan -m. File migration yang

terbentuk diisi dengan kolom sesuai ketentuan tugas.

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('laporans', function (Blueprint $table) {
            $table->id();
            $table->string('nama_pelapor');
            $table->string('no_hp', 20);
            $table->string('lokasi');
            $table->string('kecamatan');
            $table->unsignedInteger('tinggi_air');
            $table->text('keterangan')->nullable();
            $table->string('foto')->nullable();
            $table->string('status')->default('Menunggu');
            $table->timestamps();
        });
    }

    public function down(): void
    {
        Schema::dropIfExists('laporans');
    }
};

```

Kolom nama_pelapor, no_hp, lokasi, dan kecamatan bertipe string. Tinggi air disimpan sebagai unsignedInteger karena satuannya sentimeter dan tidak pernah negatif. Kolom keterangan dan foto diberi nullable() karena keduanya boleh kosong, sedangkan status diberi nilai bawaan Menunggu sehingga laporan baru langsung masuk antrean petugas. Migration dijalankan dengan php artisan migrate dan tabel laporans terbentuk di database db_laporbanjir.

Model Laporan mengikuti kolom tersebut dan menambahkan satu method kecil untuk

warna status.

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Laporan extends Model
{
    public const STATUS = ['Menunggu', 'Di proses', 'Selesai'];

    protected $fillable = [
        'nama_pelapor',
        'no_hp',
        'lokasi',
        'kecamatan',
        'tinggi_air',
        'keterangan',
        'foto',
        'status',
    ];

    public function badgeColor(): string
    {
        return match ($this->status) {
            'Di proses' => '#b45309',
            'Selesai' => '#15803d',
            default => '#b91c1c',
        };
    }
}

```

Konstanta STATUS dipakai pada pilihan status di form edit supaya daftar nilainya hanya ditulis sekali. Method badgeColor() mengembalikan warna berbeda untuk Menunggu, Di proses, dan Selesai, sehingga badge pada tabel dan halaman detail tidak perlu memakai kondisi berulang.

3.3 Membuat Controller dan Route Laporan

Controller dibuat dengan php artisan make:controller LaporanController --resource.

Bagian pertama menangani daftar laporan dan form pelaporan warga.

```php
<?php

namespace App\Http\Controllers;

use App\Models\Laporan;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Storage;

class LaporanController extends Controller
{
    public function index()
    {
        $laporans = Laporan::latest()->paginate(5);
        return view('laporan.index', compact('laporans'));
    }

    public function create()
    {
        return view('laporan.create');
    }

    public function store(Request $request)
    {
        $validated = $request->validate([
            'nama_pelapor' => 'required|string|max:255',
            'no_hp' => 'required|string|max:20',
            'lokasi' => 'required|string|max:255',
            'kecamatan' => 'required|string|max:255',
            'tinggi_air' => 'required|integer|min:0|max:1000',
            'keterangan' => 'nullable|string',
            'foto' => 'required|image|mimes:jpg,jpeg,png|max:2048',
        ]);

        $validated['foto'] = $request->file('foto')->store('foto_laporan', 'public');
        $validated['status'] = 'Menunggu';

        $laporan = Laporan::create($validated);

        return redirect()->route('laporan.berhasil', $laporan);
    }

    public function berhasil(Laporan $laporan)
    {
        return view('laporan.berhasil', compact('laporan'));
    }

```

Method index mengambil lima laporan terbaru per halaman. Method create menampilkan form pelaporan, sedangkan store memvalidasi seluruh isian lalu menyimpan foto dengan $request->file('foto')->store('foto_laporan', 'public'). Status diisi Menunggu pada saat laporan dibuat, bukan dari input warga. Setelah data tersimpan,

pengguna diarahkan ke halaman berhasil yang menampilkan nomor laporan dan status awalnya.

Bagian kedua menangani detail, form edit, proses ubah, dan hapus laporan.

```php
    public function show(Laporan $laporan)
    {
        return view('laporan.show', compact('laporan'));
    }

    public function edit(Laporan $laporan)
    {
        return view('laporan.edit', compact('laporan'));
    }

    public function update(Request $request, Laporan $laporan)
    {
        $validated = $request->validate([
            'nama_pelapor' => 'required|string|max:255',
            'no_hp' => 'required|string|max:20',
            'lokasi' => 'required|string|max:255',
            'kecamatan' => 'required|string|max:255',
            'tinggi_air' => 'required|integer|min:0|max:1000',
            'keterangan' => 'nullable|string',
            'foto' => 'nullable|image|mimes:jpg,jpeg,png|max:2048',
            'status' => 'required|in:Menunggu,Di proses,Selesai',
        ]);

        if ($request->hasFile('foto')) {
            if ($laporan->foto) {
                Storage::disk('public')->delete($laporan->foto);
            }
            $validated['foto'] = $request->file('foto')->store('foto_laporan', 'public');
        } else {
            unset($validated['foto']);
        }

        $laporan->update($validated);

        return redirect()->route('laporan.index')->with('success', 'Laporan berhasil diperbarui.');
    }

    public function destroy(Laporan $laporan)
    {
        if ($laporan->foto) {
            Storage::disk('public')->delete($laporan->foto);
        }

        $laporan->delete();

        return redirect()->route('laporan.index')->with('success', 'Laporan berhasil dihapus.');
    }
}
```

Pada method update, foto baru hanya diproses jika $request->hasFile('foto') bernilai

benar. Foto lama dihapus dengan Storage::disk('public')->delete() sebelum path baru disimpan, dan kunci foto dibuang dari data yang diupdate jika pengguna tidak memilih berkas. Method destroy menghapus foto lebih dulu, lalu menghapus baris laporannya. Validasi status memakai aturan in:Menunggu,Di proses,Selesai agar nilainya tidak bisa diisi sembarang teks.

Route dipisahkan menjadi dua kelompok. Form pelaporan warga berada di luar middleware auth, sedangkan pengelolaan laporan berada di dalamnya.

```php
<?php

use App\Http\Controllers\LaporanController;
use App\Http\Controllers\ProfileController;
use Illuminate\Support\Facades\Route;

Route::get('/', function () {
    return redirect()->route('laporan.create');
});

// Form pelaporan warga, bisa dibuka tanpa login
Route::get('/lapor', [LaporanController::class, 'create'])->name('laporan.create');
Route::post('/lapor', [LaporanController::class, 'store'])->name('laporan.store');
Route::get('/lapor/berhasil/{laporan}', [LaporanController::class, 'berhasil'])->name('laporan.berhasil');

Route::get('/dashboard', function () {
    return redirect()->route('laporan.index');
})->middleware(['auth', 'verified'])->name('dashboard');

// Halaman pengelolaan laporan oleh petugas
Route::middleware('auth')->group(function () {
    Route::get('/profile', [ProfileController::class, 'edit'])->name('profile.edit');
    Route::patch('/profile', [ProfileController::class, 'update'])->name('profile.update');
    Route::delete('/profile', [ProfileController::class, 'destroy'])->name('profile.destroy');

    Route::resource('laporan', LaporanController::class)->except(['create', 'store']);
});

require __DIR__ . '/auth.php';

```

Route GET / langsung mengarah ke form pelaporan. Tiga route /lapor dipakai warga untuk membuka form, mengirim laporan, dan melihat halaman berhasil. Route::resource('laporan', ...) dengan except(['create', 'store']) menghasilkan lima route pengelolaan, yaitu index, show, edit, update, dan destroy, karena pembuatan data sudah ditangani form warga. Route dashboard diarahkan ke daftar laporan agar

petugas yang baru login langsung melihat antrean.

3.4 Membuat Tampilan Form Pelaporan Warga

Form pelaporan tidak memakai layout Breeze karena halaman ini dibuka oleh warga yang belum punya akun. Saya membuat layout terpisah yang hanya memuat header dan satu blok gaya dasar.

```
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>@yield('title', 'LaporBanjir')</title>
    <style>
        body { font-family: Arial, Helvetica, sans-serif; background: #f3f4f6; margin: 0; color: #111827; }
        header { background: #1d4ed8; color: #ffffff; padding: 14px 20px; }
        header strong { font-size: 18px; }
        header span { font-size: 13px; }
        main { max-width: 640px; margin: 24px auto; background: #ffffff; padding: 24px; border: 1px solid #d1d5db; }
        h1 { font-size: 20px; margin-top: 0; }
        p { font-size: 14px; line-height: 1.5; }
        label { display: block; margin: 14px 0 4px; font-weight: bold; font-size: 14px; }
        input, textarea { width: 100%; padding: 8px; border: 1px solid #9ca3af; box-sizing: border-box; font: inherit; }
        textarea { height: 80px; }
        .galat { color: #b91c1c; font-size: 13px; margin-top: 4px; }
        .tombol { background: #1d4ed8; color: #ffffff; border: 0; padding: 10px 18px; margin-top: 18px; cursor: pointer; font-size: 14px; }
        table { width: 100%; border-collapse: collapse; font-size: 14px; }
        th, td { border: 1px solid #d1d5db; padding: 6px 8px; text-align: left; }
        .info { background: #ecfdf5; border: 1px solid #6ee7b7; padding: 12px; margin-bottom: 16px; }
    </style>
</head>
<body>
    <header>
        <strong>LaporBanjir</strong>
        <span>Badan Penanggulangan Bencana Daerah Kabupaten Bandung</span>
    </header>

    <main>
        @yield('content')
    </main>
</body>
</html>

```

Layout ini memakai satu blok style di dalam file, tanpa Tailwind dan tanpa proses build, sehingga halaman publik tetap ringan. Isinya hanya pengaturan huruf, lebar halaman, tabel, pesan galat, dan tombol.

Form pelaporan memuat tujuh isian sesuai kolom tabel, dengan enctype multipart/form-data.

```
@extends('layouts.publik')

@section('title', 'Lapor Banjir')

@section('content')
    <h1>Form Pelaporan Banjir</h1>
    <p>
        Isi data kejadian banjir di sekitar Anda. Laporan yang masuk akan diperiksa petugas BPBD
        dan statusnya diperbarui sampai penanganan selesai.
    </p>

    <form action="{{ route('laporan.store') }}" method="POST" enctype="multipart/form-data">
        @csrf

        <label for="nama_pelapor">Nama Pelapor</label>
        <input type="text" id="nama_pelapor" name="nama_pelapor" value="{{ old('nama_pelapor') }}">
        @error('nama_pelapor') <div class="galat">{{ $message }}</div> @enderror

        <label for="no_hp">Nomor HP</label>
        <input type="text" id="no_hp" name="no_hp" value="{{ old('no_hp') }}">
        @error('no_hp') <div class="galat">{{ $message }}</div> @enderror

        <label for="lokasi">Lokasi Kejadian</label>
        <input type="text" id="lokasi" name="lokasi" value="{{ old('lokasi') }}">
        @error('lokasi') <div class="galat">{{ $message }}</div> @enderror

        <label for="kecamatan">Kecamatan</label>
        <input type="text" id="kecamatan" name="kecamatan" value="{{ old('kecamatan') }}">

        @error('kecamatan') <div class="galat">{{ $message }}</div> @enderror

        <label for="tinggi_air">Tinggi Air (cm)</label>
        <input type="number" id="tinggi_air" name="tinggi_air" value="{{ old('tinggi_air') }}">
        @error('tinggi_air') <div class="galat">{{ $message }}</div> @enderror

        <label for="keterangan">Keterangan (opsional)</label>
        <textarea id="keterangan" name="keterangan">{{ old('keterangan') }}</textarea>
        @error('keterangan') <div class="galat">{{ $message }}</div> @enderror

        <label for="foto">Foto Kondisi Banjir (jpg, jpeg, png, maks. 2 MB)</label>
        <input type="file" id="foto" name="foto" accept="image/*">
        @error('foto') <div class="galat">{{ $message }}</div> @enderror

        <button type="submit" class="tombol">Kirim Laporan</button>
    </form>
@endsection

```

Setiap input memakai old() agar isian tidak hilang ketika validasi gagal, dan setiap input diikuti @error untuk menampilkan pesan galatnya. Input foto diberi atribut

accept=image/* sebagai petunjuk di dialog pemilihan berkas, sedangkan aturan sebenarnya tetap dijalankan di controller.

Setelah laporan tersimpan, warga melihat halaman konfirmasi berisi nomor laporan dan status awal.

```
@extends('layouts.publik')

@section('title', 'Laporan Terkirim')

@section('content')
    <h1>Laporan Berhasil Dikirim</h1>

    <div class="info">
        Terima kasih, {{ $laporan->nama_pelapor }}. Laporan Anda sudah masuk ke BPBD Kabupaten
        Bandung dengan nomor {{ $laporan->id }} dan status awal {{ $laporan->status }}.
    </div>

    <table>
        <tr>
            <th>Lokasi</th>
            <td>{{ $laporan->lokasi }}</td>
        </tr>
        <tr>
            <th>Kecamatan</th>
            <td>{{ $laporan->kecamatan }}</td>
        </tr>
        <tr>
            <th>Tinggi Air</th>
            <td>{{ $laporan->tinggi_air }} cm</td>
        </tr>
        <tr>
            <th>Status</th>
            <td>{{ $laporan->status }}</td>
        </tr>
    </table>

    <p style="margin-top: 18px">
        <a href="{{ route('laporan.create') }}">Kirim laporan lain</a>
    </p>
@endsection

```

Halaman ini menampilkan pesan terima kasih, nomor laporan, lokasi, kecamatan, tinggi air, dan status Menunggu. Nomor laporan diambil dari id baris yang baru dibuat.

3.5 Membuat Halaman Pengelolaan Laporan

Halaman pengelolaan memakai layout Breeze karena hanya dibuka petugas yang sudah login. Menu Laporan ditambahkan pada navigasi di samping Dashboard.

```
                    <x-nav-link :href="route('dashboard')" :active="request()->routeIs('dashboard')">
                        {{ __('Dashboard') }}
                    </x-nav-link>
                    <x-nav-link :href="route('laporan.index')" :active="request()->routeIs('laporan.*')">
                        {{ __('Laporan') }}
                    </x-nav-link>

```

Komponen x-nav-link dengan :active=request()->routeIs('laporan.*') membuat menu Laporan bergaris bawah saat petugas membuka salah satu halaman laporan. Komponen yang sama juga ditambahkan pada navigasi versi mobile.

Halaman daftar laporan memuat pesan berhasil, tabel data, dan tombol pagination.

```
<x-app-layout>
    <x-slot name="header">
        <h2 class="font-semibold text-xl text-gray-800 leading-tight">
            Daftar Laporan Banjir
        </h2>
    </x-slot>

    <div class="py-8">
        <div class="max-w-6xl mx-auto sm:px-6 lg:px-8">
            <div class="bg-white shadow-sm sm:rounded-lg p-6">
                @if (session('success'))
                    <div class="mb-4 p-3" style="background: #dcfce7; color: #166534;">
                        {{ session('success') }}
                    </div>
                @endif

```

Header tabel memuat sembilan kolom, termasuk kolom foto dan status.

```
<table class="tabel">
    <thead>
        <tr>
            <th>No</th>
            <th>Foto</th>
            <th>Nama Pelapor</th>
            <th>Lokasi</th>
            <th>Kecamatan</th>
            <th>Tinggi Air</th>
            <th>Status</th>
            <th>Tanggal</th>
            <th>Aksi</th>
        </tr>
    </thead>

```

Baris data diisi dengan @forelse, thumbnail foto, dan badge status berwarna.

```
                    <tbody>
                        @forelse ($laporans as $laporan)
                            <tr>
                                <td>{{ $laporans->firstItem() + $loop->index }}</td>
                                <td>
                                    @if ($laporan->foto)
                                        <img src="{{ asset('storage/' . $laporan->foto)
}}"
                                             alt="Foto laporan {{ $laporan->nama_pelapor
}}"
                                             width="70" height="50" style="object-fit:
cover;">
                                    @else
                                        <span class="text-gray-500">Tidak ada</span>
                                    @endif
                                </td>
                                <td>{{ $laporan->nama_pelapor }}</td>
                                <td>{{ $laporan->lokasi }}</td>
                                <td>{{ $laporan->kecamatan }}</td>
                                <td>{{ $laporan->tinggi_air }} cm</td>
                                <td>
                                    <span class="badge" style="background: {{ $laporan-
>badgeColor() }}">
                                        {{ $laporan->status }}
                                    </span>
                                </td>
                                <td>{{ $laporan->created_at->format('d-m-Y H:i') }}</td>
                                <td>
                                    <a href="{{ route('laporan.show', $laporan) }}"
style="color: #0369a1">Lihat</a> |
                                    <a href="{{ route('laporan.edit', $laporan) }}"
style="color: #b45309">Edit</a> |
                                    <form action="{{ route('laporan.destroy', $laporan)
}}" method="POST" class="inline">
                                        @csrf
                                        @method('DELETE')
                                        <button type="submit"
                                                style="color: #b91c1c; background: none;
border: 0; padding: 0; cursor: pointer"
                                                onclick="return confirm('Yakin ingin
menghapus laporan ini?')">
                                            Hapus
                                        </button>
                                    </form>
                                </td>
                            </tr>

```

Blok @empty dan tombol pagination menutup tabel.

```
                        @empty
                            <tr>
                                <td colspan="9" class="text-center">Belum ada laporan yang masuk.</td>
                            </tr>
                        @endforelse
                    </tbody>
                </table>

                <div class="mt-4">
                    {{ $laporans->links() }}
                </div>
            </div>
        </div>
    </div>
</x-app-layout>

```

Nomor urut memakai $laporans->firstItem() + $loop->index agar penomoran berlanjut di halaman berikutnya. Tombol Hapus mengirim request DELETE melalui @method('DELETE') dan meminta konfirmasi lewat onclick.

Halaman detail menampilkan seluruh isian laporan dalam satu tabel.

```blade
<x-app-layout>
    <x-slot name="header">
        <h2 class="font-semibold text-xl text-gray-800 leading-tight">
            Detail Laporan
        </h2>
    </x-slot>

    <div class="py-8">
        <div class="max-w-3xl mx-auto sm:px-6 lg:px-8">
            <div class="bg-white shadow-sm sm:rounded-lg p-6">
                <table class="tabel">
                    <tr>
                        <th style="width: 30%">Nama Pelapor</th>
                        <td>{{ $laporan->nama_pelapor }}</td>
                    </tr>
                    <tr>
                        <th>Nomor HP</th>
                        <td>{{ $laporan->no_hp }}</td>
                    </tr>
                    <tr>
                        <th>Lokasi</th>
                        <td>{{ $laporan->lokasi }}</td>
                    </tr>
                    <tr>
                        <th>Kecamatan</th>
                        <td>{{ $laporan->kecamatan }}</td>
                    </tr>
                    <tr>
                        <th>Tinggi Air</th>
                        <td>{{ $laporan->tinggi_air }} cm</td>
                    </tr>
                    <tr>
                        <th>Keterangan</th>
                        <td>{{ $laporan->keterangan ?? '-' }}</td>
                    </tr>
                    <tr>
                        <th>Status</th>
                        <td>
                            <span class="badge" style="background: {{ $laporan->badgeColor() }}">
                                {{ $laporan->status }}
                            </span>
                        </td>
                    </tr>
                    <tr>
                        <th>Tanggal Laporan</th>
                        <td>{{ $laporan->created_at->format('d-m-Y H:i') }}</td>
                    </tr>
                </table>

```

Bagian bawah halaman detail menampilkan foto bukti dan tombol aksi.

```
                <h3 class="font-semibold mt-6 mb-2">Foto Bukti</h3>

                @if ($laporan->foto)
                    <img src="{{ asset('storage/' . $laporan->foto) }}"
                         alt="Foto laporan {{ $laporan->nama_pelapor }}"
                         style="max-width: 420px; width: 100%; border: 1px solid
#d1d5db;">
                @else
                    <p class="text-gray-600">Laporan ini tidak memiliki foto.</p>
                @endif

                <div class="mt-6">
                    <a href="{{ route('laporan.edit', $laporan) }}"
                       class="tombol" style="background: #b45309">Edit</a>
                    <a href="{{ route('laporan.index') }}"
                       class="tombol" style="background: #4b5563">Kembali</a>
                </div>
            </div>
        </div>
    </div>
</x-app-layout>

```

Form edit memuat isian yang sama dengan form warga, ditambah pilihan status penanganan.

```blade
<x-app-layout>
    <x-slot name="header">
        <h2 class="font-semibold text-xl text-gray-800 leading-tight">
            Edit Laporan
        </h2>
    </x-slot>

    <div class="py-8">
        <div class="max-w-3xl mx-auto sm:px-6 lg:px-8">
            <div class="bg-white shadow-sm sm:rounded-lg p-6">
                <form action="{{ route('laporan.update', $laporan) }}" method="POST"
enctype="multipart/form-data">
                    @csrf
                    @method('PUT')

                    <div class="mb-4">
                        <label for="nama_pelapor" class="block text-sm font-medium">Nama
Pelapor</label>
                        <input type="text" id="nama_pelapor" name="nama_pelapor"
                               value="{{ old('nama_pelapor', $laporan->nama_pelapor) }}"
                               class="w-full border p-2">
                        @error('nama_pelapor') <p class="text-red-600 text-sm">{{ $message
}}</p> @enderror
                    </div>

                    <div class="mb-4">
                        <label for="no_hp" class="block text-sm font-medium">Nomor
HP</label>
                        <input type="text" id="no_hp" name="no_hp"
                               value="{{ old('no_hp', $laporan->no_hp) }}"
                               class="w-full border p-2">
                        @error('no_hp') <p class="text-red-600 text-sm">{{ $message }}</p>
@enderror
                    </div>

                    <div class="mb-4">
                        <label for="lokasi" class="block text-sm font-
medium">Lokasi</label>
                        <input type="text" id="lokasi" name="lokasi"
                               value="{{ old('lokasi', $laporan->lokasi) }}"
                               class="w-full border p-2">
                        @error('lokasi') <p class="text-red-600 text-sm">{{ $message }}
</p> @enderror
                    </div>

                    <div class="mb-4">
                        <label for="kecamatan" class="block text-sm font-
medium">Kecamatan</label>
                        <input type="text" id="kecamatan" name="kecamatan"
                               value="{{ old('kecamatan', $laporan->kecamatan) }}"
                               class="w-full border p-2">
                        @error('kecamatan') <p class="text-red-600 text-sm">{{ $message }}
</p> @enderror
                    </div>

```

Pilihan status memakai konstanta STATUS pada model, sehingga daftar nilainya sama dengan validasi.

```
                    <div class="mb-4">
                        <label for="tinggi_air" class="block text-sm font-medium">Tinggi Air (cm)</label>
                        <input type="number" id="tinggi_air" name="tinggi_air"
                               value="{{ old('tinggi_air', $laporan->tinggi_air) }}"
                               class="w-full border p-2">
                        @error('tinggi_air') <p class="text-red-600 text-sm">{{ $message }}</p> @enderror
                    </div>

                    <div class="mb-4">
                        <label for="keterangan" class="block text-sm font-medium">Keterangan</label>
                        <textarea id="keterangan" name="keterangan" rows="3"
                                  class="w-full border p-2">{{ old('keterangan', $laporan->keterangan) }}</textarea>
                        @error('keterangan') <p class="text-red-600 text-sm">{{ $message }}</p> @enderror
                    </div>

                    <div class="mb-4">
                        <label for="status" class="block text-sm font-medium">Status Penanganan</label>
                        <select id="status" name="status" class="w-full border p-2">
                            @foreach (App\Models\Laporan::STATUS as $status)
                                <option value="{{ $status }}" @selected(old('status', $laporan->status) === $status)>
                                    {{ $status }}
                                </option>
                            @endforeach
                        </select>
                        @error('status') <p class="text-red-600 text-sm">{{ $message }}</p> @enderror
                    </div>

                    <div class="mb-4">
                        <p class="text-sm">Foto saat ini:</p>
                        @if ($laporan->foto)
                            <img src="{{ asset('storage/' . $laporan->foto) }}"
                                 alt="Foto laporan {{ $laporan->nama_pelapor }}"
                                 width="160" style="border: 1px solid #d1d5db;">
                        @else

```

Bagian foto menampilkan gambar yang sedang tersimpan dan input penggantian yang bersifat opsional.

```
                            <p class="text-gray-600 text-sm">Belum ada foto.</p>
                        @endif
                        <label for="foto" class="block text-sm font-medium mt-2">
                            Ganti Foto (kosongkan jika tidak diganti, maks. 2 MB)
                        </label>
                        <input type="file" id="foto" name="foto" accept="image/*"
class="w-full">
                        @error('foto') <p class="text-red-600 text-sm">{{ $message }}</p>
@enderror
                    </div>

                    <button type="submit" class="tombol" style="background: #1d4ed8;
border: 0;">
                        Simpan Perubahan
                    </button>
                    <a href="{{ route('laporan.index') }}" class="ms-3 text-gray-
600">Batal</a>
                </form>
            </div>
        </div>
    </div>
</x-app-layout>

```

3.6 Menambahkan Gaya Dasar

Tampilan dibuat sederhana karena yang dibutuhkan hanya halaman yang bisa dipakai, bukan desain khusus. Saya menambahkan lima aturan CSS pada resources/css/app.css untuk tabel, badge status, dan tombol, lalu memakai kelas tersebut di view.

```css
@tailwind base;
@tailwind components;
@tailwind utilities;

/* Gaya dasar untuk halaman pengelolaan laporan */
.tabel {
    width: 100%;
    border-collapse: collapse;
    font-size: 14px;
}

.tabel th,
.tabel td {
    border: 1px solid #d1d5db;
    padding: 6px;
    text-align: left;
}

.tabel thead {
    background-color: #f3f4f6;
}

.badge {
    color: #ffffff;
    padding: 2px 6px;
    font-size: 12px;
}

.tombol {
    color: #ffffff;
    padding: 8px 14px;
    text-decoration: none;
    display: inline-block;
}

```

Aturan ini ikut terkompilasi bersama Tailwind melalui npm run build. Setelah view baru ditambahkan, asset perlu di-build ulang agar kelas yang hanya dipakai di view

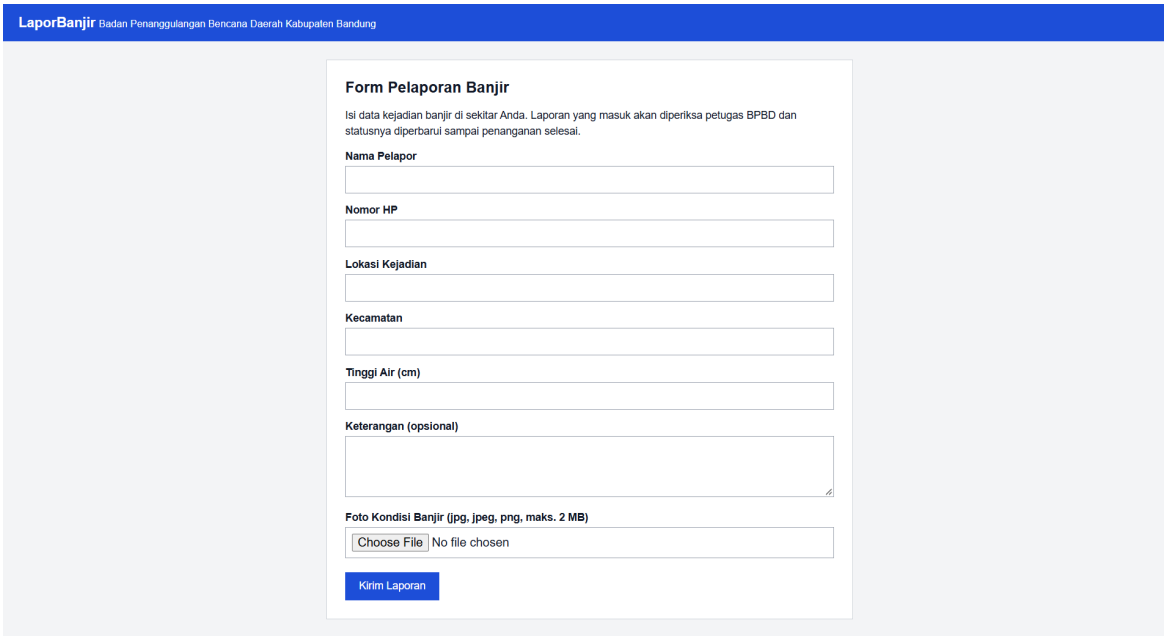
baru ikut masuk ke file CSS hasil build.

3.7 Menguji Alur Pelaporan dan Pengelolaan

Pengujian dikerjakan langsung di browser pada alamat http://127.0.0.1:8125. Form pelaporan diuji dengan isian kosong, berkas ZIP, gambar 6 MB, dan data yang benar. Sisi petugas diuji dengan membuka /laporan sebelum login, mendaftarkan akun petugas, lalu mencoba daftar, detail, edit status, ganti foto, dan hapus laporan. Enam laporan dibuat melalui form warga agar pagination terlihat, kemudian satu laporan dihapus sehingga data akhir berisi lima laporan.

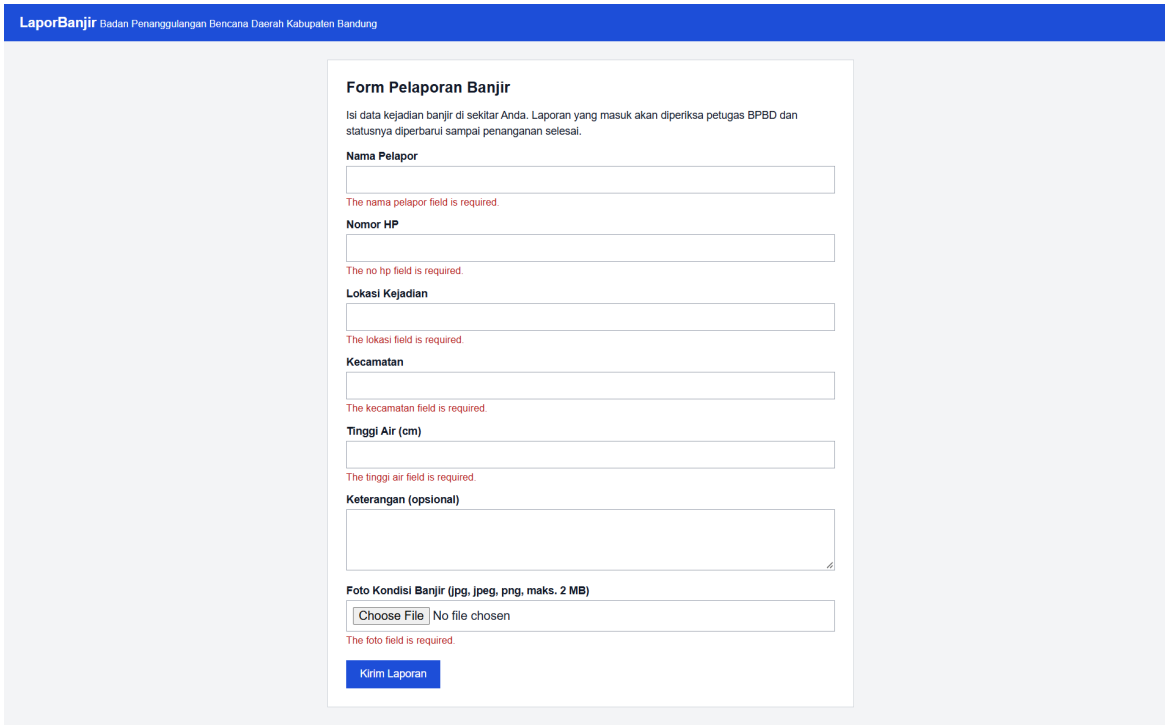
4. Hasil dan Dokumentasi Program

Berikut adalah tangkapan layar aplikasi LaporBanjir yang berjalan melalui php artisan serve pada alamat http://127.0.0.1:8125. Halaman publik memakai layout sederhana, sedangkan halaman petugas memakai layout Breeze.



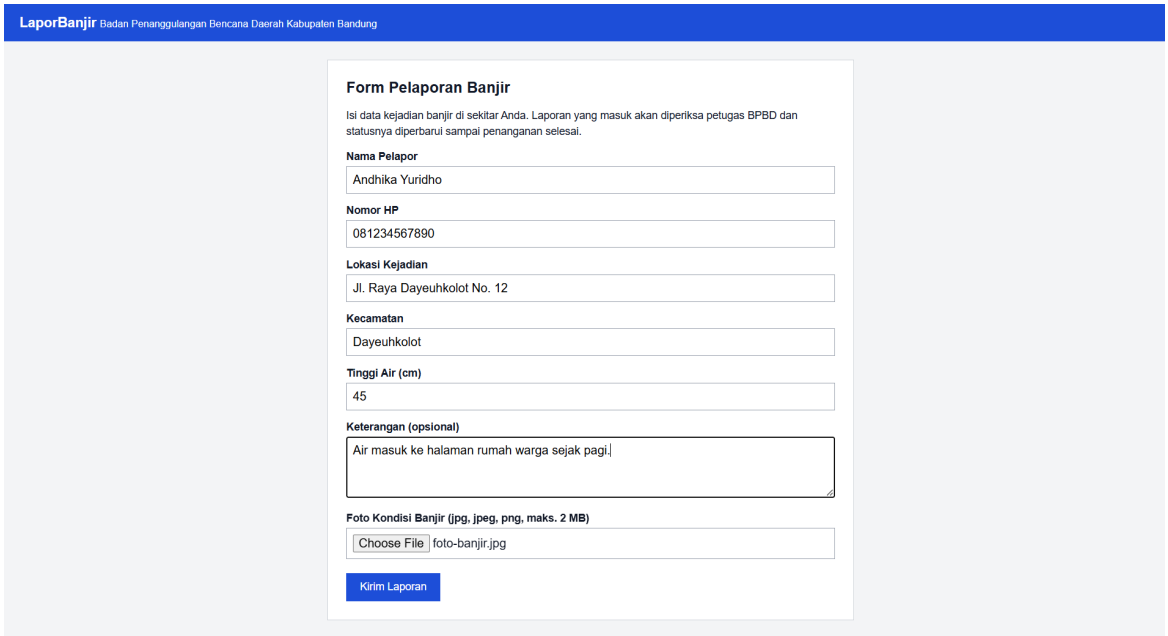
Gambar 1. Form pelaporan pada alamat /lapor

Form dapat dibuka tanpa login dan memuat tujuh isian, yaitu nama pelapor, nomor HP, lokasi, kecamatan, tinggi air, keterangan, dan foto kondisi banjir.



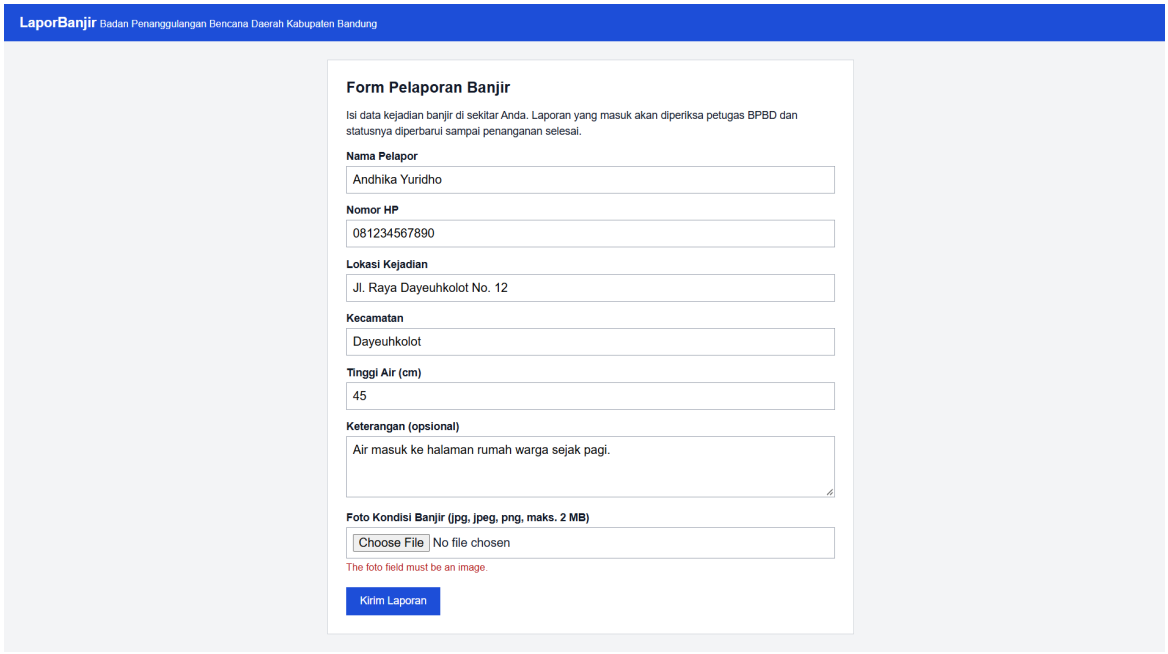
Gambar 2. Pesan galat ketika form dikirim kosong

Setiap input wajib menampilkan pesannya sendiri di bawah kolom yang bersangkutan, mulai dari "The nama pelapor field is required." sampai "The foto field is required."



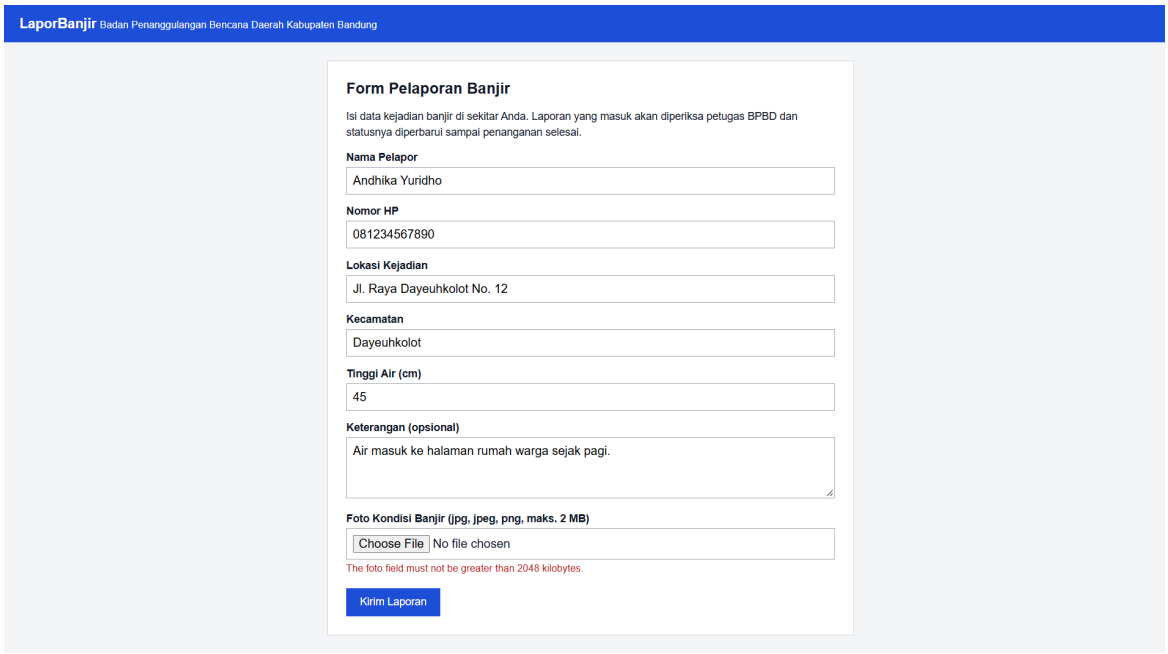
Gambar 3. Form pelaporan yang sudah diisi data warga

Form diisi atas nama Andhika Yuridho untuk lokasi Jl. Raya Dayeuhkolot No. 12 dengan tinggi air 45 cm dan satu foto JPG berukuran 1,6 MB.



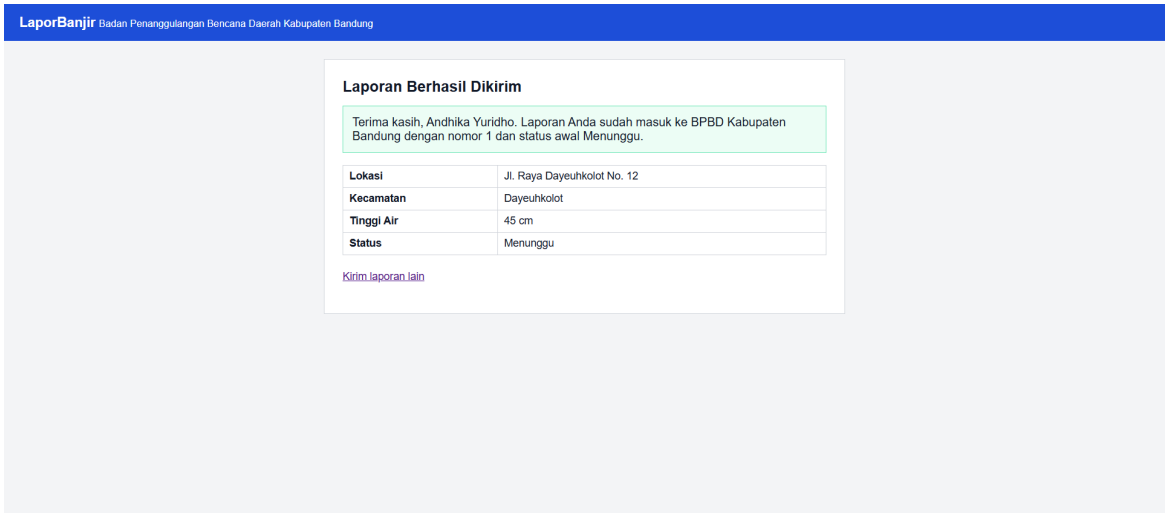
Gambar 4. Pesan galat ketika berkas yang dipilih bukan gambar

Berkas arsip.zip ditolak aturan image dan mimes. Pesan "The foto field must be an image." muncul di bawah input foto, sedangkan isian lain tetap terisi.



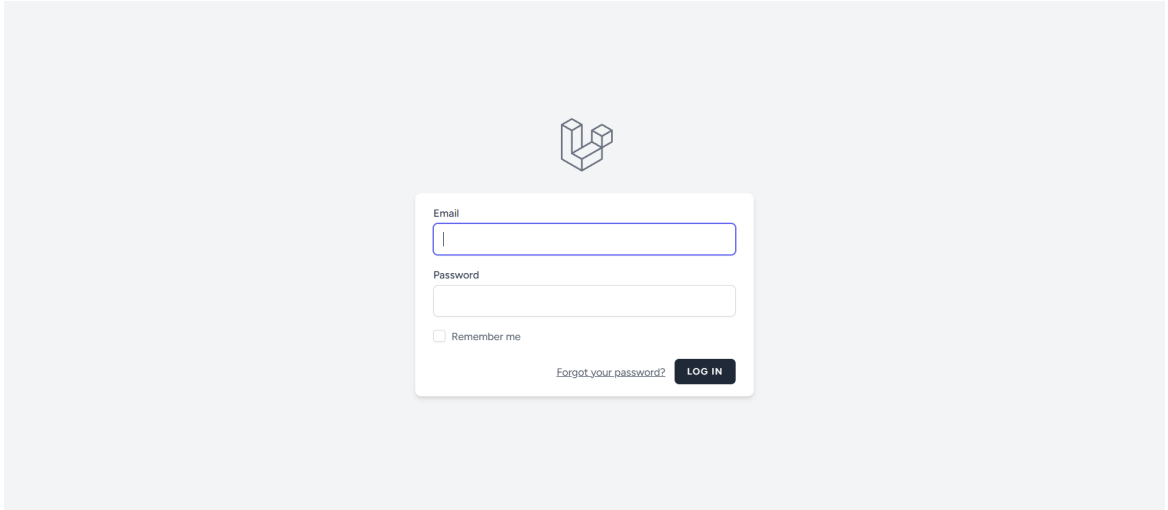
Gambar 5. Pesan galat ketika foto melebihi 2 MB

Gambar berukuran 6 MB ditolak aturan max:2048 dengan pesan "The foto field must not be greater than 2048 kilobytes."



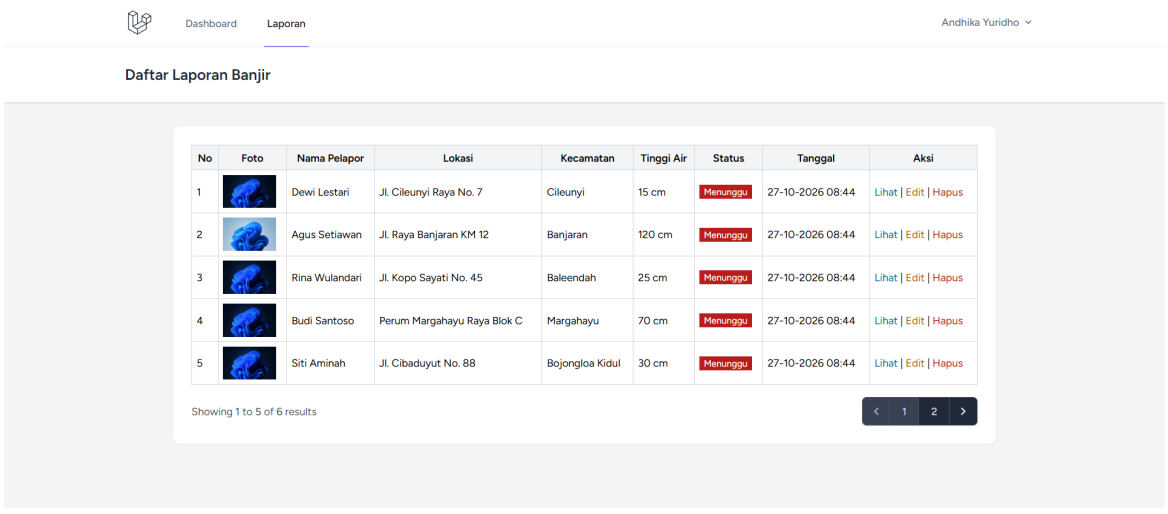
Gambar 6. Halaman konfirmasi setelah laporan dikirim

Laporan pertama tersimpan dengan nomor 1 dan status Menunggu. Halaman ini menampilkan ringkasan lokasi, kecamatan, tinggi air, dan status awal laporan.



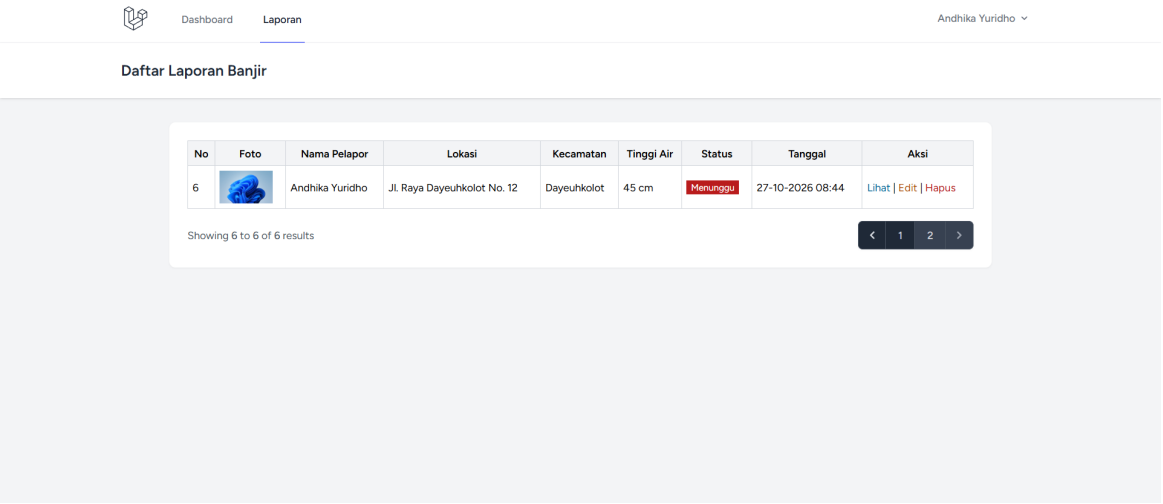
Gambar 7. Halaman login saat /laporan dibuka sebelum login

Middleware auth mengalihkan permintaan ke halaman login Breeze. Daftar laporan tidak sempat tampil sebelum petugas masuk.



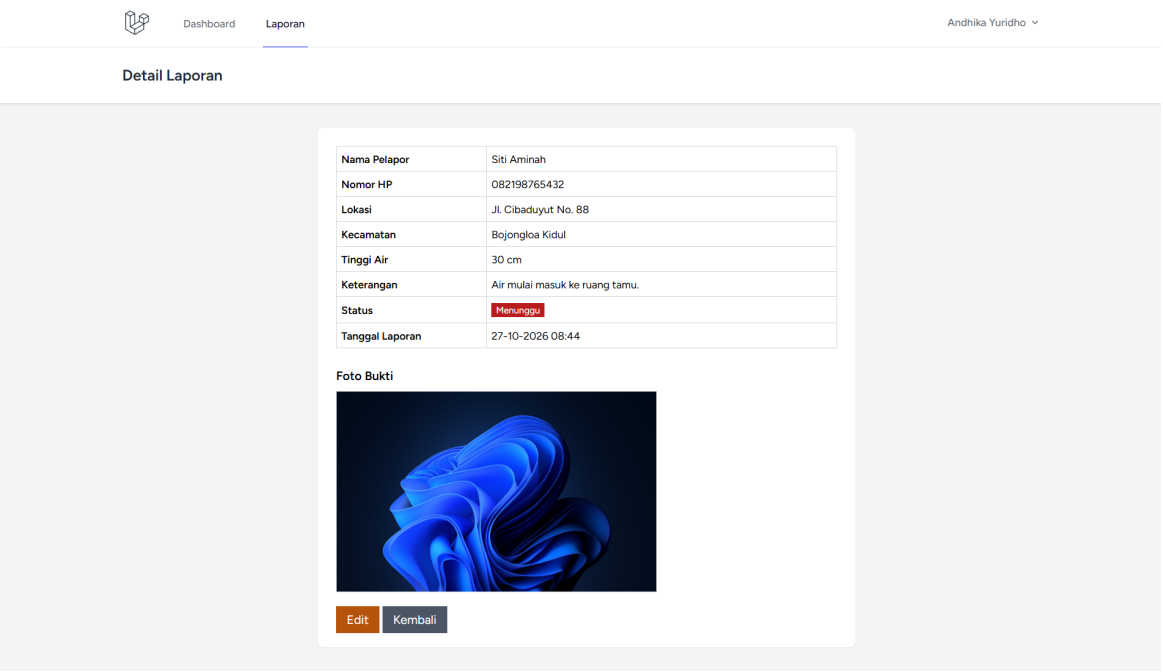
Gambar 8. Daftar laporan halaman 1 dengan thumbnail dan pagination

Enam laporan hasil pengujian ditampilkan lima per halaman. Setiap baris memuat thumbnail foto, nama pelapor, lokasi, kecamatan, tinggi air, badge status, tanggal, dan tombol aksi.



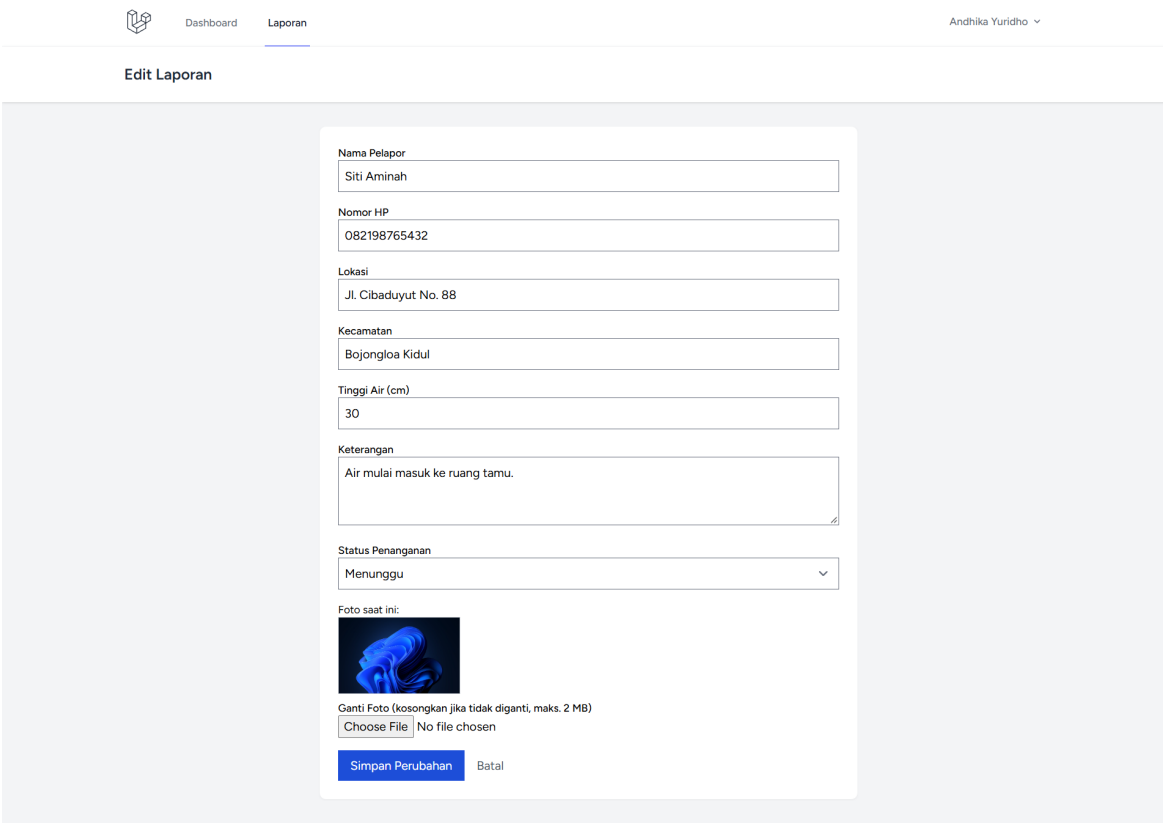
Gambar 9. Daftar laporan halaman 2

Halaman kedua berisi satu laporan dengan nomor urut 6, jadi penomoran berlanjut karena memakai firstItem() ditambah indeks perulangan.



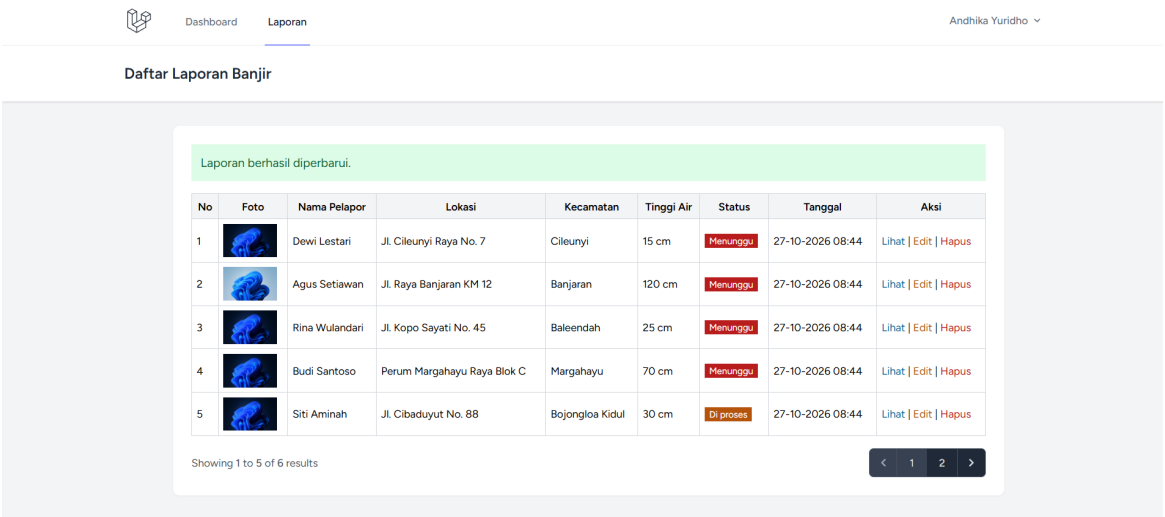
Gambar 10. Halaman detail laporan

Halaman detail menampilkan seluruh isian laporan beserta foto bukti berukuran 420 piksel dan badge status Menunggu.



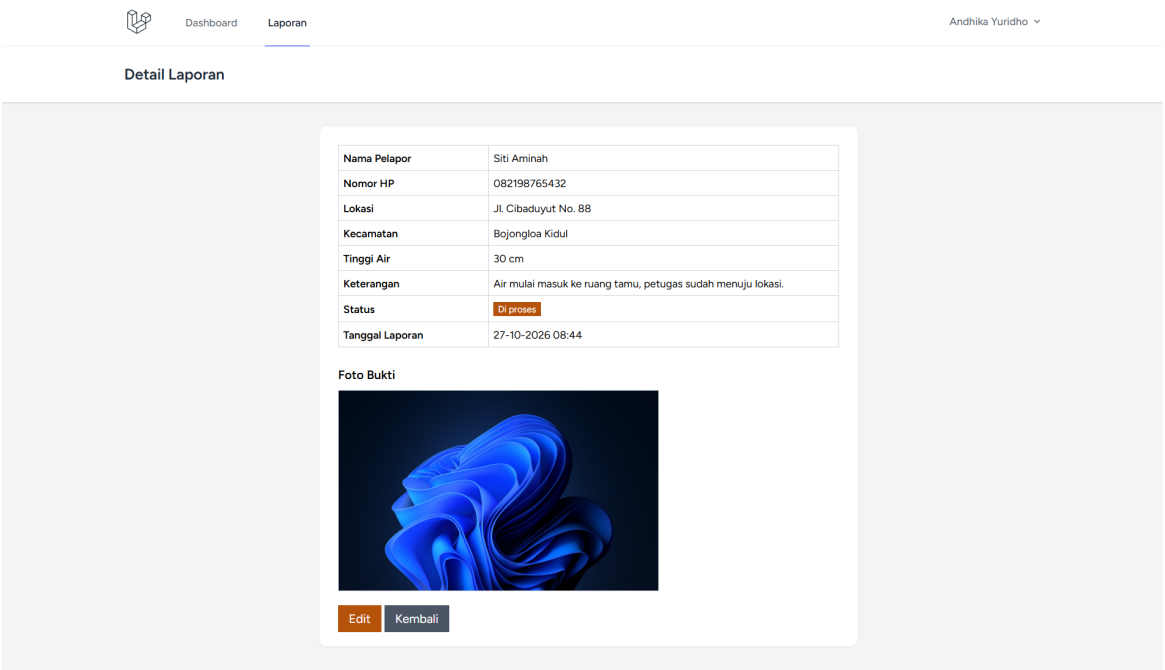
Gambar 11. Form edit laporan sebelum diubah

Form menampilkan data lama, pilihan status penanganan, dan foto yang sedang tersimpan. Input foto diberi keterangan boleh dikosongkan jika tidak ingin mengganti.



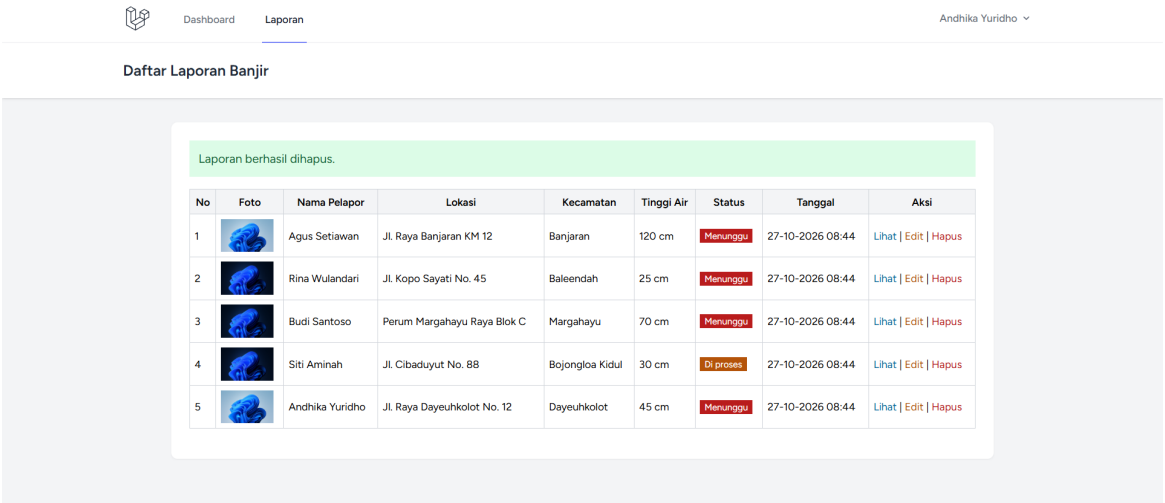
Gambar 12. Pesan berhasil setelah laporan diperbarui

Status laporan Siti Aminah diubah menjadi Di proses dan fotonya diganti. Pesan "Laporan berhasil diperbarui." tampil di atas tabel dan badge pada baris tersebut berubah warna.



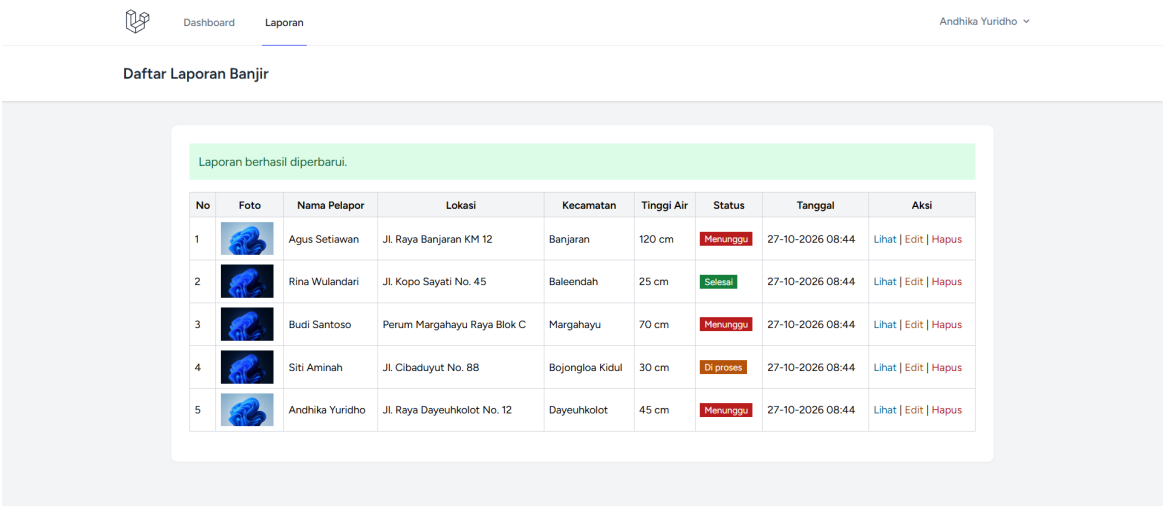
Gambar 13. Detail laporan setelah status dan foto diubah

Halaman detail menampilkan status Di proses dan foto pengganti. Foto lama sudah dihapus dari folder storage pada saat proses update.



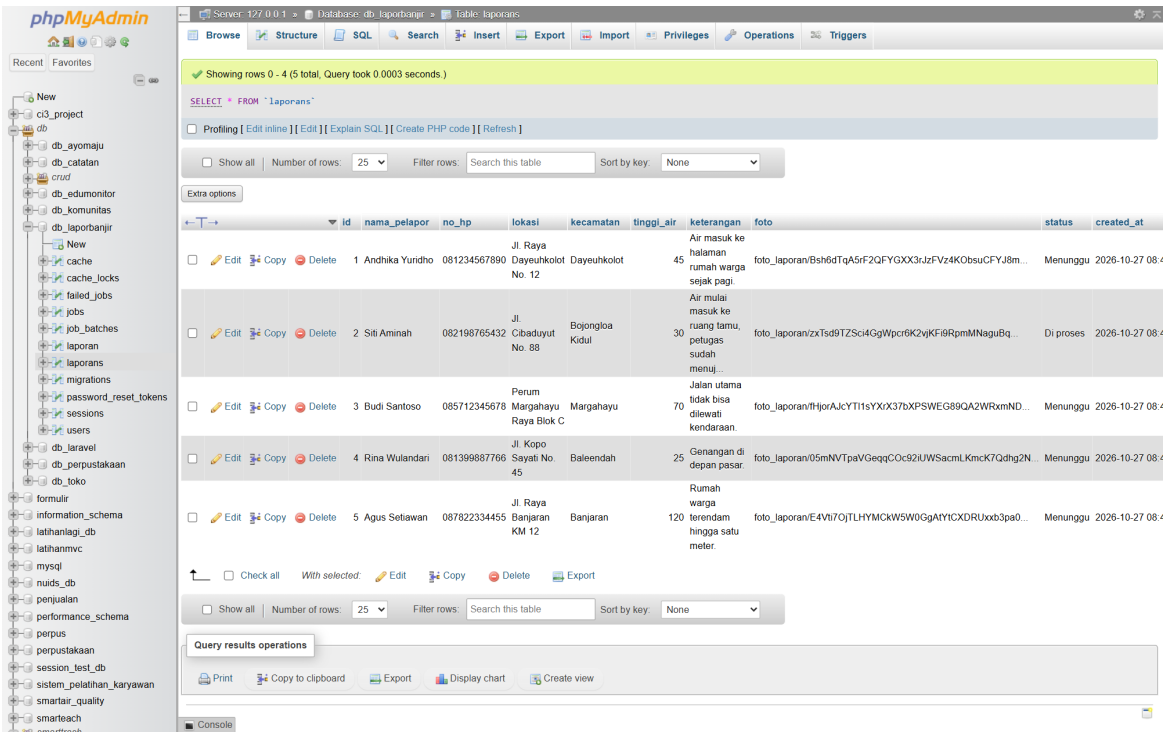
Gambar 14. Pesan berhasil setelah laporan dihapus

Laporan Dewi Lestari dihapus setelah konfirmasi. Pesan "Laporan berhasil dihapus." tampil di atas tabel dan jumlah data berkurang menjadi lima.



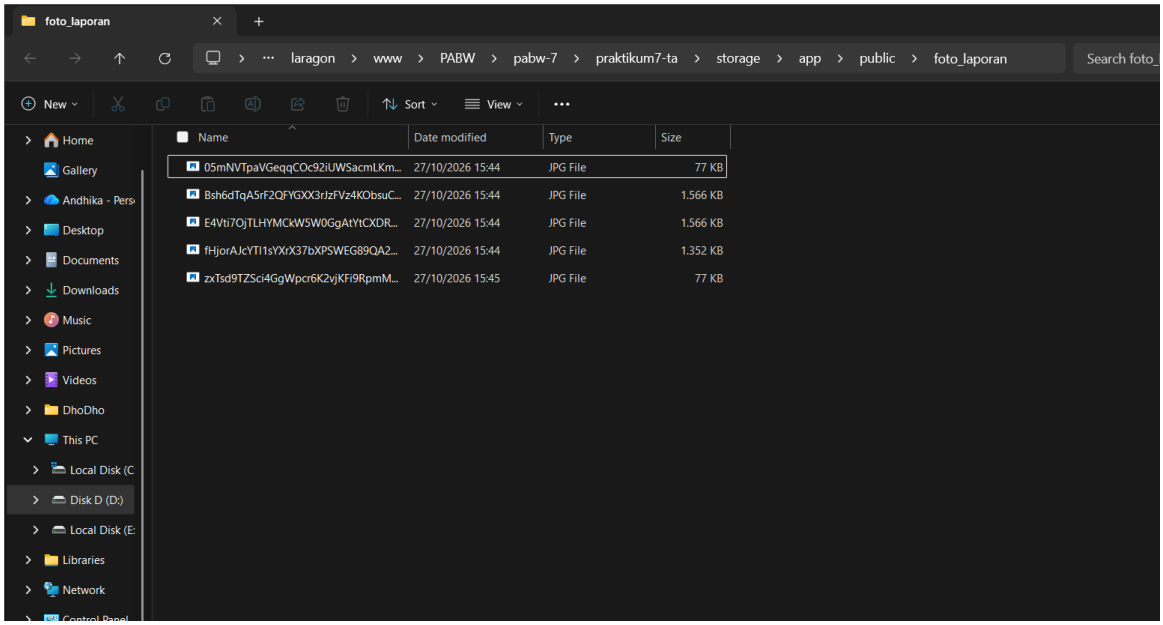
Gambar 15. Daftar laporan dengan tiga nilai status

Setelah satu laporan diselesaikan, tabel menampilkan status Menunggu, Di proses, dan Selesai sekaligus dengan warna badge yang berbeda.



Gambar 16. Isi tabel laporans pada phpMyAdmin

Tabel menyimpan identitas pelapor, lokasi, kecamatan, tinggi air, keterangan, path foto, dan status. Kolom foto berisi path seperti foto_laporan/Bsh6dTqA5rF2QFYGXX3rJzFVz4KObsuCFYJ8mEmz.jpg, bukan isi gambarnya.



Gambar 17. Isi folder storage/app/public/foto_laporan

Folder berisi lima berkas gambar dengan nama acak, sama banyak dengan jumlah laporan yang tersisa. Tidak ada berkas sisa dari foto yang diganti maupun laporan yang dihapus.

5. Analisis dan Pembahasan

Pembagian route menjadi dua kelompok menentukan siapa yang boleh melakukan apa. Tiga route /lapor berada di luar middleware auth karena warga tidak punya akun, sedangkan lima route pengelolaan berada di dalamnya. Dengan pembagian ini, satu controller bisa menangani dua peran sekaligus: method create dan store dipakai warga, sedangkan index, show, edit, update, dan destroy hanya bisa dicapai setelah login. Pengujian pada Gambar 7 menunjukkan permintaan ke /laporan tanpa session langsung dialihkan ke /login, dan setelah akun petugas didaftarkan pengguna diarahkan ke daftar laporan.

Resource route memotong pekerjaan pendaftaran route. Satu baris Route::resource dengan except(['create', 'store']) menghasilkan lima route beserta nama, method HTTP, dan parameter yang baku, dan route model binding mengisi parameter Laporan $laporan dari id pada URL. Karena pembuatan laporan sudah ditangani form warga, dua method resource yang tidak dipakai dikeluarkan lewat except() supaya tidak ada route /laporan/create yang tidak pernah dibuka.

Pemisahan isi berkas dan path di database terlihat pada hasil pengujian. Tabel laporans hanya menyimpan path hasil store() pada kolom foto, sedangkan gambarnya berada di storage/app/public/foto_laporan seperti pada Gambar 17. Nama berkas di storage berupa string acak, sehingga dua warga yang mengunggah foto dengan nama sama tidak saling menimpa. Folder storage berada di luar document root, jadi public/storage harus ada sebagai symbolic link agar asset('storage/...') bisa menampilkan gambar.

Konsistensi antara database dan storage dijaga di dua tempat. Method update menghapus foto lama sebelum menyimpan path baru, dan method destroy menghapus foto sebelum menghapus barisnya. Setelah enam laporan dibuat, satu laporan diganti fotonya, dan satu laporan dihapus, tabel berisi lima baris dan folder foto_laporan berisi lima berkas. Jumlah keduanya sama, sehingga tidak ada berkas yang tertinggal tanpa pemilik.

Validasi foto bekerja pada dua lapis aturan. Aturan image dan mimes:jpg,jpeg,png menolak arsip ZIP seperti pada Gambar 4, dan aturan max:2048 menolak gambar 6 MB seperti pada Gambar 5. Pesan galat yang muncul masih pesan bawaan Laravel dalam bahasa Inggris karena tidak ada file bahasa Indonesia maupun argumen pesan khusus pada method validate(). Pesan itu tetap tampil di bawah input yang bersalah, sesuai ketentuan tugas.

Status penanganan disimpan sebagai teks dengan nilai bawaan Menunggu. Pilihan status di form edit dibatasi tiga nilai, dan validasi in:Menunggu,Di proses,Selesai menjaga agar nilai lain tidak masuk lewat request manual. Warna badge diambil dari method badgeColor() pada model, sehingga tampilan status di tabel, halaman detail, dan halaman edit selalu sama.

Tampilan sengaja dibuat sederhana. Halaman publik memakai satu blok CSS di dalam layout tanpa Tailwind, sedangkan halaman petugas memakai komponen Breeze dan empat aturan CSS tambahan pada resources/css/app.css. Cara ini membuat halaman publik tidak bergantung pada hasil build, tetapi halaman petugas tetap memerlukan npm run build setiap kali kelas baru dipakai di view.

Beberapa penyesuaian saya lakukan terhadap teks tugas. Folder project tetap bernama praktikum7-ta, bukan laporbanjir, karena folder itu yang disiapkan untuk tugas. Tabel baru bernama laporans, bukan laporan, karena database db_laporbanjir masih menyimpan tabel laporan dari tugas Modul 6 dengan skema lama, dan tabel itu tidak saya ubah. Validasi pada tugas juga meminta foto wajib berupa gambar, jadi input foto saya buat required pada form warga dan nullable pada form edit karena foto lama sudah ada.

Kendala yang muncul lebih banyak pada lingkungan kerja daripada pada kode aplikasi. Laragon tidak bisa dijalankan dari terminal karena meminta hak administrator, sehingga MySQL dan Apache saya nyalakan langsung dari file executable-nya sebelum pengujian. Setelah view baru dibuat, asset juga harus di-build ulang agar kelas CSS yang hanya dipakai view baru ikut terkompilasi. Selama pengujian, file storage/logs/laravel.log bahkan belum terbentuk sama sekali, artinya tidak ada galat aplikasi yang tercatat dan tidak ada kegagalan yang perlu dibahas di sini.

6. Kesimpulan

Aplikasi LaporBanjir berhasil dibangun dari project Laravel baru dengan Laravel Breeze. Warga dapat mengirim laporan beserta foto bukti tanpa login, dan petugas dapat melihat daftar laporan dengan thumbnail dan pagination, membuka detail, mengubah status penanganan, mengganti foto, serta menghapus laporan. Seluruh halaman pengelolaan terlindungi middleware auth.

Saya memahami bahwa upload file memerlukan tiga hal yang saling terkait: atribut enctype pada form, validasi jenis dan ukuran berkas di controller, dan symbolic link public/storage agar berkas bisa ditampilkan. Berkas disimpan di storage dan database hanya menyimpan path-nya, sehingga penghapusan foto lama harus dilakukan bersamaan dengan penggantian path agar tidak ada berkas yang tertinggal.

Pemakaian Breeze mempercepat bagian authentication dan navigasi, sedangkan seluruh logika CRUD dan upload tetap ditulis sendiri di controller. Tampilan sederhana dengan lima aturan CSS tambahan sudah cukup untuk kebutuhan pengelolaan data

pada tugas ini.

7. Link Repository Github

https://github.com/dhodhoo/pabw-andhika-yuridho/tree/main/pabw-7/praktikum7-ta